

Contents

1	CASD spark/hdfs cluster onboarding	1
1.1	1. Architecture of the cluster	1
1.2	2. Check web interface (TS-D1MUTUA)	2
1.3	3. Check the access of d1mutua-client	2
1.3.1	3.1 Get your username	2
1.3.2	3.2 Connect to the d1mutua-client server	2
1.3.3	3.3 Transfer data between TS-D1MUTUA and d1mutua-client	3
1.3.4	3.4 Create shortcut for ssh and scp	3
1.3.5	3.5 Test the new command	4
1.4	4. Check the hdfs client access	4
1.4.1	4.1 transfer data from local file system to hdfs cluster	5
1.5	5. Check the spark client access	5
1.5.1	5.1 Use spark cluster in interactive mode	6

1 CASD spark/hdfs cluster onboarding

This tutorial aims to help you to be familiar on how to use spark/hdfs cluster inside CASD infrastructure.

We will follow the below steps in this tutorial.

- understand the general architecture of the cluster
- check web interface access
- config and check ssh access
- config and check hdfs client access
- config and check spark client access

1.1 1. Architecture of the cluster

The below figure shows the general architecture of the cluster

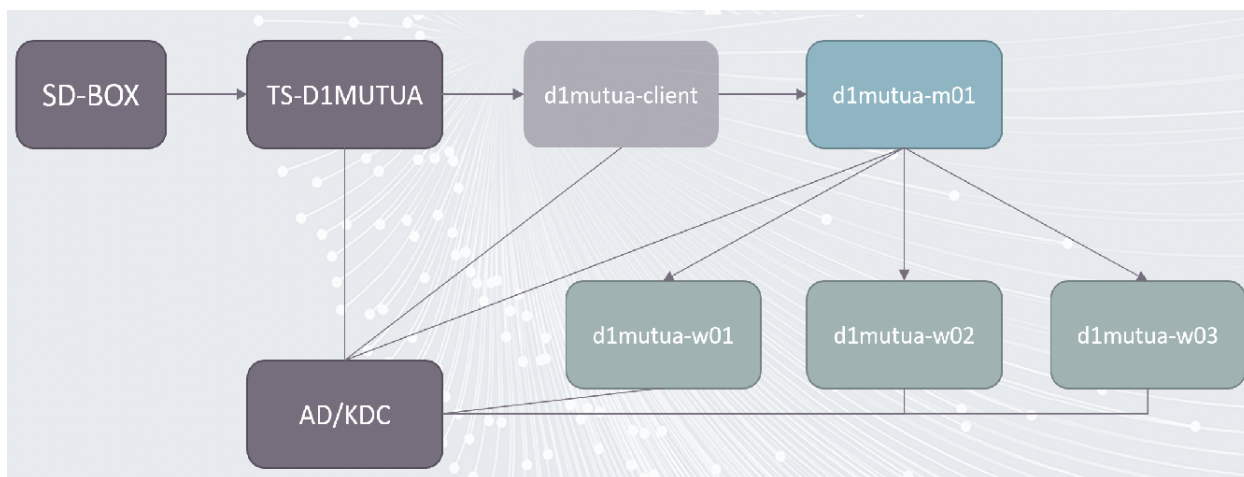


Figure 1: architecture.png

- The TS-D1MUTUA: is the main server(Windows) which you connect to via the SD-BOX.
- The d1mutua-client: is the server(debian 13) which allows you to interact with the spark/hdfs cluster.
- The d1mutua-m01, d1mutua-w01/w02/w03: are the servers which form the spark/hdfs cluster. The end users can not access them directly, but through spark/hdfs client which are installed/configured on d1mutua-client.

1.2 2. Check web interface (TS-D1MUTUA)

This steps happens inside the TS-D1MUTUA server(Windows).

You can visit the following url by using the **Chrome** web browser.

- <https://d1mutua-m01.casd.fr:50470/explorer.html> : is the hdfs cluster web interface which allows you to view directories and data of your projects.
- <https://d1mutua-m01.casd.fr:8090/cluster> : is the spark cluster web interface which allows you to view spark jobs and available resources of the cluster.

The first connection may take few seconds, because of the ticket and group checking. You may also need to accept the certificat if you see warnings.

1.3 3. Check the access of d1mutua-client

This steps happens inside the TS-D1MUTUA server(Windows). It allows you to test if you can connect to the d1mutua-client server.

1.3.1 3.1 Get your username

Get the right username is very important for the rest of this tutorial, it specifies which is your login name for:

- d1mutua-client server.
- directory and file access control on the hdfs cluster
- spark job resources control

To get your username:

- open a powershell terminal
- type the below command

```
$env:USERNAME
```

```
# for example, the output for me  
D1MUTUA_P_LIU0000
```

The username must be in capital letter.

1.3.2 3.2 Connect to the d1mutua-client server

As we mentioned before, to use the spark/hdfs cluster, you need to connect to the d1mutua-clientserver(Linux). The server access is done via **ssh** protocol.

To test the connectivity:

- open a powershell terminal.
- type the below command

```
# general form of the command  
ssh -K <username>@d1mutua-client.casd.fr
```

```
# for example, for me the command should be like  
ssh -K D1MUTUA_P_LIU0000@d1mutua-client.casd.fr
```

After the above command, you should see a welcoming message from CASD. You should also notice the header of the powershell terminal changed from to. From now on, all the commands you entered inside this terminal will be executed on the d1mutua-client server(Linux).

For now, we only allow end users to access d1mutua-client server. All the other servers of the cluster are not accessible.

1.3.3 3.3 Transfer data between TS-D1MUTUA and d1mutua-client

Suppose you have data(i.e. test.txt) in TS-D1MUTUA(Windows), you want to transfer the data to d1mutua-client(Linux):

- Open a new powershell terminal
- Run the below command

```
# go to the test data folder
```

```
cd C:\Users\Public\Documents\hadoop_cluster_onboarding\data
```

```
# for example, transfer a file test.txt, don't forget to replace D1MUTUA_P_LIU0000 by your username
```

```
scp -o GSSAPIAuthentication=yes t1.txt D1MUTUA_P_LIU0000@d1mutua-client.casd.fr:/home/D1MUTUA_P_LIU0000
```

```
# for example, transfer a folder, you need to add -r to make the command recursive
```

```
scp -o GSSAPIAuthentication=yes -r folder1/ D1MUTUA_P_LIU0000@d1mutua-client.casd.fr:/home/D1MUTUA_P_LIU0000
```

To run the above command in your context, you need to replace the D1MUTUA_P_LIU0000 by your username(check section 3.1) and you need to have t1.txt and folder1 in TS-D1MUTUA(Windows).

Below is the general form of the scp command.

```
# general form of the command
```

```
scp -o GSSAPIAuthentication=yes <src_data> <USERNAME>@d1mutua-client.casd.fr:/home/<USERNAME>/
```

To check if the data arrived correctly, you can use the terminal opened in step 3.2 Connect to the d1mutua-client server In this terminal, you are in d1mutua-client(Linux).

```
# go back to your home folder in d1mutua-client
```

```
cd
```

```
# list all files and directories
```

```
ls
```

```
# disconnect from the `d1mutua-client` server
```

```
exit
```

you should see the test.txt and data_folder. If everything is ok, you can close all the terminal now.

1.3.4 3.4 Create shortcut for ssh and scp

To facilitate your access to the d1mutua-client server(Linux) and data transfer, CASD have developed a little script which can generate ssh config files to make the ssh command shorter

Open a new powershell terminal, and type the below command

```
# goto the script folder
```

```
cd C:\Users\Public\Documents\hadoop_cluster_onboarding\scripts
```

```
# run the script
```

```
.\gen_ssh_conf.ps1
```

```
# expected output example
```

```
# SSH Config created and secured for D1MUTUA_P_LIU0000
```

This script creates an ssh config file which allows you to do the ssh without typing your username and a new command kscp which allows you to copy data to the d1mutua-client server more easily. You can close this terminal now.

1.3.5 3.5 Test the new command

To test the new command, you need to open a new `powershell` terminal.

1.3.5.1 3.5.1 Connect to the `d1mutua-client` server Now you can use the below command to connect to the `d1mutua-client` server

```
ssh d1mutua-client.casd.fr
```

```
# if everything works well, you should see the welcome message of CASD
```

```
# disconnect from the `d1mutua-client` server
```

```
exit
```

1.3.5.2 3.5.2 Use `kscp` to transfer data The new command `kscp` allows us to transfer data with fewer arguments.

Reuse the `powershell` terminal opened in section 3.5.1, then enter the below command

```
# go to the test data folder
```

```
cd C:\Users\Public\Documents\hadoop_cluster_onboarding\data
```

```
# copy a single file
```

```
kscp stats.csv d1mutua-client.casd.fr
```

```
# copy a folder
```

```
kscp -r folder2 d1mutua-client.casd.fr
```

If you want to customize the `kscp` command, below shows the general form.

```
kscp <src_data> <USERNAME>@d1mutua-client.casd.fr:/home/<USERNAME>/
```

After the data transfer, the data arrive to the server `d1mutua-client.casd.fr`. If you want to check the data, you need to connect to the `d1mutua-client` server(Linux).

```
# connect to the `d1mutua-client` server
```

```
ssh d1mutua-client.casd.fr
```

```
# check the data
```

```
ls
```

```
# disconnect
```

```
exit
```

You can close this terminal if everything works well

1.4 4. Check the hdfs client access

For now, we only support `hdfs client` under Linux, so you need to `ssh` to the `d1mutua-client.casd.fr` server first.

Open a `powershell` terminal, and run the below command

```
# connect to the `d1mutua-client` server
```

```
ssh d1mutua-client.casd.fr
```

You should see a welcome message from CASD. From now on, as you are in the `d1mutua-client` server(Linux). The `powershell` command will not work anymore.

In `d1mutua-client` server(Linux), you have two different file system. - local file system of `d1mutua-client` server. - distributed file system of the hdfs cluster.

Run the below command to test your hdfs client.

```
# check the hdfs file system
hdfs dfs -ls /
```

```
# check user home folder, $USER will be replaced by the current user name
hdfs dfs -ls /users/$USER
```

```
# expected output for user D1MUTUA_P_LIU0000
drwx-----+ - D1MUTUA_P_LIU0000 hadoop          0 2026-03-11 09:06 /users/D1MUTUA_P_LIU0000/.sparkStag
-rw-----+  3 D1MUTUA_P_LIU0000 hadoop        76 2026-03-06 11:18 /users/D1MUTUA_P_LIU0000/stats.csv
drwx-----+ - D1MUTUA_P_LIU0000 hadoop          0 2026-03-10 17:16 /users/D1MUTUA_P_LIU0000/tmp
```

You can also try to access the project folder

```
# check the projects folder
hdfs dfs -ls /projects
```

```
# try to access a project
hdfs dfs -ls /projects/BCL_EEC
```

1.4.1 4.1 transfer data from local file system to hdfs cluster

As `spark` is a framework for distributed computing, the data must be also distributed. That's why `spark` can't use data from the local file system of `d1mutua-client` server. As a result, we must transfer data from local file system to hdfs cluster

Use the same terminal of section 4. The below commands run inside `d1mutua-client` server.

```
# go to your home directory of `d1mutua-client` server
cd
# check data in the local file system
ls
```

```
# you should see stats.csv in the output. We copied in section 3.5.2
```

```
# now upload data from `local file system` to `hdfs cluster`
hdfs dfs -put stats.csv /users/$USER/
```

```
# check the data in hdfs
hdfs dfs -ls /users/$USER
```

1.5 5. Check the spark client access

First check if you have `spark` installed in your environment. Use the same terminal of section 4.1. The below commands run inside `d1mutua-client` server.

```
# check current spark runtime version
spark-submit --version
```

```
# check if you have kerberos ticket
klist
```

```
# create your first spark job
nano job1.py
```

Put the below code in the job1.py file

```
from pyspark.sql import SparkSession

def main():
    # Create Spark session
    spark = SparkSession.builder \
        .appName("pengfei_test") \
        .getOrCreate()

    # For a basic test, create a small DataFrame
    df = spark.createDataFrame([
        ("Alice", 25),
        ("Bob", 30),
        ("Charlie", 35)
    ], ["name", "age"])

    df.show()

    # Just for validation: print row count
    print(f"Total rows: {df.count()}")

    spark.stop()

if __name__ == "__main__":
    main()
```

use ctrl+o for saving file. ctrl+x for exiting.

Now we can submit the job to the cluster with the below command

```
spark-submit --name=pengfei_test_job job1.py
```

By default, we have configured the spark client in mode cluster. So nothing runs in d1mutua-client.casd.fr. As a result, you don't need to install python environment and pyspark You can check the status of your job via yarn web UI.

1.5.1 5.1 Use spark cluster in interactive mode

We have seen how to submit job to the cluster. For exploring data, you may want to your spark job interactively(client mode). In this mode the spark driver runs on d1mutua-client.casd.fr. So we need to install a python virtual environment and pyspark

Use the same terminal of section 5. The below commands run inside d1mutua-client server.

```
# check your python version
python3 -V
```

```
# create a virtual env
python3 -m venv spark_venv
```

```
# activate the python venv
source spark_venv/bin/activate
```

```
# check installed libs
pip list
```

```
# install pyspark, the pyspark version must match the spark version in the cluster
```

```
pip install pyspark==3.5.7
```

```
# install jupyterlab  
pip install jupyterlab
```

To facilitate the usage of jupyterlab, CASD has developed a launcher to avoid port conflict between users. To start a jupyterlab, you can run

```
run_jupyterlab
```

```
# expected output
```

```
INFO: Using port: 8888
```

```
INFO: JupyterLab server runs with URL: http://d1mutua-client:8888/lab?token=79b0a30a9a2fc6adae67...482d
```

```
INFO: To stop the JupyterLab, use ctrl+C or close the terminal.
```

You need to copy the jupyterlab url and open it with a browser in TS-D1MUTUA.